

Aetna Insurance Virtual Assistant

Retrieval-Augmented Generation (RAG) Chatbot — End-to-End Process Guide

Frontend	Streamlit
Backend	Python
LLM	OpenAI GPT-4o
Embedding Model	text-embedding-3-large
Orchestration Framework	LangChain
Vector Database	ChromaDB
Document Parsing	PyMuPDF

This document walks through the complete 10-step pipeline that powers the chatbot: from a user uploading a PDF insurance document, all the way to a grounded, cited answer appearing in the chat window. It maps every step to the exact Python module that implements it.

Pipeline at a Glance

The pipeline has two phases: **Ingestion** (Steps 1-5, runs once per uploaded document) and **Question Answering** (Steps 6-10, runs once per user question).

INGESTION PHASE (runs when documents are uploaded)

1. Upload PDF → 2. Extract Text → 3. Chunk Text → 4. Generate Embeddings → 5. Store in ChromaDB

QUESTION ANSWERING PHASE (runs on every user question)

6. Embed Question → 7. Similarity Search → 8. Build Prompt w/ Retrieved Chunks → 9. LLM Generates Answer → 10. Display + Cite

Detailed Step-by-Step Breakdown

Step 1 — Upload PDF Files

Module: `frontend/streamlit_ui.py`

The user uploads one or more insurance PDFs (Evidence of Coverage, Summary of Benefits and Coverage, Medicare Managed Care Manual, etc.) through a Streamlit file-uploader widget in the sidebar. Uploaded files are saved to the `data/pdfs/` folder on disk so downstream steps (which use PyMuPDF, a disk-based PDF reader) can process them.

Step 2 — Extract Text from PDFs

Module: `ingestion/pdf_loader.py`

PyMuPDF (imported as 'fitz') opens each PDF and extracts plain text page by page. Each page's text is tagged with metadata — the source filename and page number — which travels with that text all the way through the pipeline and ultimately powers citations in Step 10.

Step 3 — Chunk Documents

Module: `chunking/text_splitter.py`

Each page's text is split into smaller, overlapping chunks (default: 1000 characters, 200-character overlap) using LangChain's `RecursiveCharacterTextSplitter`. Chunking keeps each embedded piece of text focused on one topic, which makes similarity search much more precise than embedding an entire page or document at once. Overlap prevents a single important fact (e.g., a dollar amount) from being split across two chunks and lost.

Step 4 — Generate Embeddings

Module: `embeddings/embedding_service.py`

Each text chunk is converted into a numeric vector using OpenAI's text-embedding-3-large model, via LangChain's `OpenAIEmbeddings` wrapper. An embedding captures the MEANING of text — so a question like 'how much do I pay before coverage starts?' can match a chunk that says 'annual deductible' even though they share no exact keywords.

Step 5 — Store Embeddings in Vector Database

Module: `vectorstore/chroma_manager.py`

Every chunk's vector, original text, and metadata (source + page) are stored together in ChromaDB, an open-source vector database that persists to a local folder (`data/chroma_db/`). This turns the uploaded documents into a searchable knowledge base.

Step 6 — Perform Similarity Search

Module: `retrieval/retriever.py`

When a user asks a question, the question itself is embedded using the SAME embedding model used on the documents (critical — mixing embedding models produces meaningless similarity scores). ChromaDB then compares the question's vector against every stored chunk vector to find the closest matches.

Step 7 — Retrieve Top Matching Chunks

Module: retrieval/retriever.py

The top-K most similar chunks (default K=5, configurable via .env) are returned, ranked by similarity, each carrying its source filename and page number forward for later citation.

Step 8 — Send Retrieved Chunks to the LLM

Module: prompts/prompt_template.py

The retrieved chunks are formatted into a clearly labeled CONTEXT block and combined with the user's question and a strict system prompt instructing GPT-4o to answer ONLY from the provided context, to cite every fact with (Source: filename, page X), and to say 'I don't know' rather than guess when the context is insufficient.

Step 9 — Generate a Grounded Response

Module: rag/rag_pipeline.py

The combined system + user messages are sent to OpenAI's GPT-4o (via LangChain's ChatOpenAI, temperature=0 for factual, deterministic output). GPT-4o synthesizes the retrieved chunks into a clear, plain-language answer, expanding on insurance jargon only where the retrieved text supports it.

Step 10 — Display Answer with Citations

Module: frontend/streamlit_ui.py

The generated answer is rendered as a chat bubble in the Streamlit UI, with an expandable 'Sources' panel underneath listing every retrieved chunk's document name, page number, and a text preview — so the user can verify the chatbot's answer directly against the original PDF.

Why the Chatbot Only Answers from Uploaded Documents

Insurance questions about deductibles, copays, coverage limits, appeal deadlines, and prior authorization requirements can have real financial and health consequences if answered incorrectly. This assistant is deliberately designed to be 'grounded': it never relies on the LLM's general training knowledge about insurance in general, and never guesses. It only answers using text retrieved from the specific documents the user uploaded, and cites every claim back to a source document and page number so the user can verify it themselves.

If the uploaded documents don't contain enough information to answer a question, the assistant is instructed to say so plainly and direct the user to Aetna Member Services or their official plan documents, rather than fabricating an answer.

Disclaimer

This assistant summarizes uploaded plan documents. It does not provide legal, medical, or financial advice, and does not make coverage decisions.